

מערכות הפעלה – תרגיל 1

לתרגיל זה 3 חלקים:

- א. כתיבת קובץ סקריפט לתרגול פקודות לינוקס.
 - ב. פיתוח shell פשוט בשפת C עם קריאות המערכת שנלמדו.
 - ג. שאלות מילוליות.
- בסוף התרגיל מפורטות הוראות ההגשה והנחיות כלליות. יש לקרוא אותן בעיון ולהקפיד על כל ההנחיות!

חלק א

עליכם ליצור קובץ סקריפט בשם `os1.sh`.

הסקריפט מקבל שלושה ארגומנטים ומבצע את הפעולות הבאות (לפי הסדר):

1. יוצר תיקייה חדשה בתיקיית העבודה הנוכחית, עם שם לפי הארגומנט הראשון. את כל הקבצים והפעולות בהמשך הסקריפט יש ליצור ולבצע בתוך תיקייה זו.
2. יוצר קובץ חדש בשם `greeting.txt` עם התוכן הבא: `Hey AAA! My name is BBB` כאשר את AAA יש להחליף עם שם המשתמש הנוכחי, ואת BBB עם שמכם המלא. (רמז: משתנה הסביבה `USER` מכיל את שם המשתמש הנוכחי)
3. מעתיק את הקובץ הנתון בארגומנט השני אל תוך התיקייה, ואז מקמפל אותו אל קובץ הריצה `os1exe` עם כל האזהרות כפי שנלמד. (רמז: בהינתן נתיב מלא `X`, הסימון `$(basename X)` מחזיר את שם הקובץ של `X` ללא התיקיות)
4. מדפיס את רשימת הקבצים בתיקייה הנתונה בארגומנט השלישי (רמז: הדגלים `-la`).

שימו לב: כל צעד 1-4 בסקריפט צריך להיות מבוצע **בשורה אחת בלבד**. ניתן להניח שמתקבל מספר נכון של ארגומנטים והם תקינים, הקבצים הנדרשים כבר קיימים, התיקייה שיש ליצור לא קיימת מראש, יש את ההרשאות המתאימות, פרמטרים 2,3 הם נתיב מלא, וכו'.

כמו-כן, שימו לב לעבוד בעורך טקסט על קובץ עם ירידות שורה של Unix:

- ב-Notepad++ אפשר לבצע זאת עם: `Edit → EOL Conversion → Unix (LF)`
- ב-VSCode אפשר לבצע זאת ע"י לחיצה על הכיתוב `CRLF` בפינה הימנית התחתונה של החלון, ובחירה באפשרות `LF` (אם כתוב `LF` – זה כבר מעודכן נכון).

מערכות הפעלה – תרגיל 1

חלק ב

עליכם לכתוב תוכנית C בשם `os1.c`. התוכנית תממש shell פשוט אשר מריץ פקודות משתמש ותומך בהרצת פקודות ברקע (ללא זומבים).

ה-shell רץ בלולאה, כאשר בכל איטרציה נדפיס "\$\$" (דולר-דולר-רווח), נקרא פקודה אחת מהמשתמש, ונריץ אותה בתהליך חדש. במידה והמשתמש בחר להריץ את הפקודה ברקע (תו % בסוף הפקודה) נמשיך מיד לאיטרציה הבאה, אחרת נמתין שהתהליך שנוצר יסתיים לפני שנמשיך לאיטרציה הבאה.

התוכנית תסתיים כאשר אין יותר קלט (EOF). ניתן לדמות זאת ע"י הרצה עם קלט מקובץ (redirection), או ע"י צירוף המקשים CTRL+D אשר גורם ל-EOF בקריאת הקלט ע"י התוכנית שרצה כרגע.

- כדי לקרוא קלט מהמשתמש יש להשתמש בפקודה הבאה: `fgets(line, 2048, stdin)`

פקודה זו קוראת שורה מהמשתמש אל תוך מערך `char` בשם `line`, ומחזירה NULL במקרה של EOF. ניתן להניח שכל שורת קלט אינה חורגת מ-2046 תווים (2048 כולל ירידת שורה ותו סיום מחזורית).

- כדי לפצל מחזורית למילים (פיצול שורת הקלט למערך ארגומנטים), יש להשתמש בקטע הקוד הבא:

```
#include <string.h>
#define STR_SEP " \t\n"

char* args[200];
args[0] = strtok(line, STR_SEP);

int i = 0;
while (args[i] != NULL) {
    ++i;
    args[i] = strtok(NULL, STR_SEP);
}
```

בסיומו, המערך `args` יכיל את מילה אחת בכל תא והמשתנה `i` יכיל את מספר המילים (כמות הארגומנטים, כולל שם התוכנית). התא `args[i]` יכיל NULL. ניתן להניח ששורת הקלט אינה מכילה יותר מ-199 מילים ולהתעלם מטיפול במרכאות (ארגומנטים עם כמה מילים).

- הרצה ברקע תסומן ע"י % בסוף השורה. אין צורך להתייחס לאף סימון מיוחד אחר (כמו שרשור או הפניה), מספר מופעים של %, וכו'. בהרצה ברקע (ובכל התוכנית) יש לוודא שאף זומבי לא נשאר לצמיתות, כך שכל זומבי נאסף עד הפקודה הבאה של המשתמש (כמו shell סטנדרטי).

בסיום, התוכנית תדפיס ירידת שורה.

שימו לב:

- במימוש התוכנית יש להשתמש בקריאות מערכת **בלבד** ולא בפונקציות ספריה של שפת C, מלבד `printf`, `perror` ואלו שסופקו בקוד לדוגמה. יש לבדוק את ערכי ההחזר של כל קריאות המערכת.
- על הפלט שלכם להיות **זהה לחלוטין** לפלט הנדרש. מצורף לתרגיל פתרון מקומפל שתוכלו להיעזר בו כדי להשוות פלט, ע"י הפניית פלט (redirection) והשוואה בין 2 הקבצים. הפקודה `diff` תעזור לבצע השוואה מדויקת: `diff f1 f2` (אם אין פלט לפקודה – הקבצים `f1`, `f2` זהים).
- כאשר מריצים תהליכים ברקע, באופן טבעי יתכנו הבדלים בסדר ההדפסות בין הקוד שלכם לפתרון המקומפל, ואף בין ריצות שונות של אותו קוד. זה תקין, והבדיקה תתמקד באופן פעולה נכון ופלט נכון ולא בסדר הדפסות שזהה בדיוק לפתרון (במקרה של הרצה ברקע).
- כאשר נקראת פונקציה לטיפול בסיגנל, במידה והתהליך נמצא במהלך קריאת מערכת היא תיכשל (לדוגמה: קריאת `wait` תחזור אפילו אם לא הסתיים תהליך בן). לכן, כדי להימנע מזומבים עדיף להשתמש בפקודה: `waitpid(-1, NULL, WNOHANG)`. פקודה זו "אוספת" תהליך בן **אחד** שהסתיים ומחזירה את ה-PID שלו **מבלי להמתין**. אם אין כזה, הפקודה מחזירה מיד 0 או -1.
- בכל פעם שמסתיים תהליך **שהורץ ברקע**, על ה-shell להדפיס את מזהה התהליך שלו (את מזהה התהליך בלבד, ללא אף פלט נוסף ובשורה נפרדת).

מערכות הפעלה – תרגיל 1

חלק ג

נתונה התוכנית הבאה:

```
1  int main() {
2      int r1 = !fork() || fork();
3      if (r1) {
4          char* args[3] = {"echo", "r1: TRUE", NULL};
5          execvp(args[0], args);
6          printf("r1: TRUE\n");
7      } else {
8          printf("r1: FALSE\n");
9      }
10     wait(NULL);
11
12     int r2 = !fork() && fork();
13     if (r2) {
14         char* args[3] = {"echo", "r2: TRUE", NULL};
15         execvp(args[0], args);
16     }
17     else {
18         printf("r2: FALSE\n");
19     }
20     if (r1 || r2)
21         printf("END\n");
22 }
```

תזכורת: באופרטורים &&, ||, אם התוצאה ידועה לפי האופרנד הראשון, לא נבדק/מבוצע האופרנד השני.

1. כתבו פלט אפשרי של התוכנית הנתונה בהנחה שכל קריאות המערכת הצליחו, וציירו את עץ התהליכים שנוצר בעקבות הריצה שמובילה לפלט זה (יש לציין לכל תהליך בעץ את מזהה התהליך שלו). יש להניח שמזהה התהליך הראשי (שמריץ את main) הוא 100, וכל תהליך שנוצר מקבל מזהה הגדול ב-1 מהתהליך שנוצר לפניו.

2. כתבו פלט אפשרי של התוכנית הנתונה בהנחה שכל הקריאות ל-fork הצליחו ו**הקובץ echo לא קיים במערכת**, וציירו את עץ התהליכים שנוצר בעקבות הריצה שמובילה לפלט זה (יש לציין לכל תהליך בעץ את מזהה התהליך שלו). יש להניח שמזהה התהליך הראשי (שמריץ את main) הוא 200, וכל תהליך שנוצר מקבל מזהה הגדול ב-1 מהתהליך שנוצר לפניו.

את התשובות יש להגיש במסמך **PDF (בלבד!)** בשם **os1.pdf**. מסמך ה-PDF צריך להיות **מוקלד** (ללא כתב יד! ניתן לצרף צילומים/ציורים של עצי התהליכים).

מערכות הפעלה – תרגיל 1

הוראות הגשה:

- יש להגיש את התרגיל בקובץ tar.gz **יחיד**. שם הקובץ הוא ת"ז בלבד (9 ספרות) וסיומת. לדוגמה, אם הת"ז הוא 123456789 אז שם הקובץ להגשה יהיה **123456789.tar.gz**
- הקובץ יכול את 3 קבצי התרגיל **בלבד**, ללא תיקיות או קבצים נסתרים. את הקובץ יש ליצור ב-VM באמצעות הפקודה הבאה:
tar czf <yourid>.tar.gz os1.sh os1.c os1.pdf
- מצורף לתרגיל סקריפט פייתון (בשם checksub.py). יש להריץ אותו עם שם קובץ ההגשה כפרמטר, לבדיקת תקינות ההגשה. **לא יתקבלו ערעורים על הגשה לא תקינה.**

הנחיות כלליות:

- ההגשה היא **ביחידים**. כל דמיון בין הגשות **ולו הקטן ביותר** יועבר מיידית לטיפול ועדת המשמעת של המכללה ללא אזהרה או אפשרות ערעור. מקורות אונליין (כולל LLM כמו ChatGPT) שקולים להעתקה. פתרו את התרגיל **באופן עצמאי לחלוטין** והימנעו מבעיות.
- הציון על שאלות הקוד מתמקד בנכונות הקוד (קומפילציה ללא אזהרות, בדיקה וטיפול בשגיאות, ניהול משאבים וזיכרון, וכו') ולא עיצוב הקוד (תיעוד, חלוקה לפונקציות, וכו'). עם זאת, יש לכתוב קוד קריא וברור. הקוד יקומפל באמצעות הפקודה: **gcc <codefiles> -Wall -o <exefile>**
- בכל ערעור על קוד יש לצרף תמונת מסך של הטרמינל עם הפלט של פקודת **hostnamectl**, ביצוע **gcc** לקוד, והרצה של הקוד עם פלט תקין – מתוך VM על גבי **vagrant** כפי שנלמד. מי שלא פועל לפי ההנחיות (מערכת הפעלה שונה, או ללא **vagrant**) – **מוותר על הזכות לערער**.
- **הגשות באיחור** יתקבלו עם הפחתה של 10 נק' לכל יום איחור מעבר למועד ההגשה, אשר מסתיים בדיוק **בשעה 23:00**. דאגו להגיש מספיק זמן מראש, לא יתקבל אישור על פספוס ברגע האחרון. במקרה של מחלה, מילואים, או כל סיבה מוצדקת אחרת, יש לשלוח אישור אל המרצה **לפחות 24 שעות** לפני ההגשה.